

COMP551 Assignment1

Yuhui Wu, Yuran Wang, Zhibo Wang

February 2026

1 Abstract

This project investigates a real-world regression task using the UCI Bike Sharing Dataset, with the goal of predicting the total number of bikes rented per day. We follow a complete machine learning workflow including data pre-processing, exploratory analysis, model implementation, and evaluation. The dataset is inspected for data quality issues, and features that are irrelevant or lead to data leakage are removed. Categorical variables are encoded using one-hot encoding, and continuous features are standardized to support stable optimization.

Linear regression is implemented from scratch using the analytical closed-form least squares solution, avoiding explicit matrix inversion for numerical stability. The dataset is split into training and test sets, and performance is evaluated using mean squared error (MSE). In addition to the baseline model, simple non-linear feature engineering techniques are explored to analyze their impact on model behavior.

The results show that the baseline linear regression captures general demand trends but remains limited in modeling complex variability. Introducing non-linear feature engineering reduces prediction error on both training and test sets; however, the improvement is more pronounced on the training data, highlighting the trade-off between increased model capacity and generalization.

2 Introduction

This project studies a real-world regression problem using the UCI Bike Sharing Dataset, with the aim of predicting the total number of bikes rented per day (cnt) [1]. The data set includes weather, temporal, and seasonal features that are expected to influence daily rental demand. We begin by loading, cleaning, and exploring the data to identify potential data quality issues and to understand feature distributions and their relationships with the target vari-

able. During preprocessing, particular attention is paid to handling categorical features appropriately, removing irrelevant variables, and avoiding data leakage [2]. We then implement linear regression from scratch using the analytical closed-form solution, relying on numerically stable least-squares computation rather than explicit matrix inversion. The dataset is split into training and test sets to ensure unbiased evaluation, and model performance is measured using mean squared error (MSE). Finally, we introduce simple non-linear feature engineering techniques to examine their effect on model capacity and training performance [2]. Through this process, the project aims to deepen understanding of the strengths and limitations of linear regression when applied to real-world data.

3 Data Preprocessing and Exploration

We first inspected the dataset structure, including the number of samples, feature types, and potential missing values. A check for NaN values was performed, and no missing entries were found in the dataset. Since the data was complete and well-structured, no imputation or row removal was required.

We then removed features that are irrelevant or could introduce data leakage. Specifically, “instant” (record index) and “dteday” (date string) were excluded because they do not provide meaningful predictive information in this context. More importantly, “casual” and “registered” were removed because their sum directly equals the target variable “cnt”. Including them would artificially inflate performance and result in data leakage.

Categorical features, including season, year, month, holiday, weekday, working day, and weather situation, were converted into numerical form using one-hot encoding. This ensures that the model does not incorrectly assume ordinal relationships between category values. The continuous variables (temperature, feeling temperature, humidity, and wind speed) were retained as numerical features. Although these variables were already scaled between 0 and 1 in the original dataset, we later applied standardization after splitting the training and test sets in Task 2 to improve numerical stability and avoid data leakage.

For exploratory analysis, we generated histograms of selected numerical features and scatter plots between continuous variables and the target variable. The scatter plots revealed a positive relationship between temperature and bike rentals, indicating that higher temperatures are associated with increased demand. Humidity and wind speed showed weaker negative relationships. The distribution of the target variable suggests moderate skewness but no extreme outliers. These observations motivated our later exploration of nonlinear feature transformations.

It is worth mentioning that we went beyond basic analysis and **creatively used heat maps** for further analysis.

4 Methods

Linear Regression Model

We implement a linear regression model from scratch using Python and NumPy. The model parameters are estimated using the analytical closed-form solution of least squares, with a bias term explicitly included in the design matrix. To ensure numerical stability, we avoid explicit matrix inversion during optimization and instead employ NumPy’s least-squares solver, which internally relies on singular value decomposition (SVD).

Encapsulating the model in a class structure with fit and predict methods allowed us to maintain clear organization and reproducibility across experiments. Linear regression was selected as a baseline model because of its interpretability and analytical tractability, making it suitable for analyzing the effects of preprocessing and feature transformations.

Train/Test Split and Evaluation

After preprocessing, the dataset was randomly divided into training and test sets using a fixed random seed. The test set was strictly reserved for evaluation and was not used in computing scaling statistics or model parameters. In particular, feature standardization was performed using only training set statistics to prevent data leakage [3]. Model performance was evaluated using mean squared error (MSE), which measures the average squared difference between predicted and actual values. Reporting both training and test MSE allows us to assess model fit as well as generalization performance.

Non-linear Feature Engineering

To explore whether simple nonlinear relationships could improve model fit, we augmented the original feature set with additional transformed features. Specifically, we introduced squared terms for selected continuous variables to capture potential curvature in the relationship between weather conditions and bike demand. We also added interaction terms between selected feature pairs to model combined effects.

Although these transformations increase model flexibility, the resulting model remains linear in its parameters. Therefore, the optimization procedure remains unchanged. By comparing training performance before and after feature expansion, we can assess whether increasing model capacity leads to improved fit and whether the original linear assumptions were too restrictive.

5 Results

The baseline linear regression model trained on the original feature set achieves a training mean squared error (MSE) of 555,437.2357 and a test MSE

of 658,413.2054. The relatively large magnitude of the MSE values is expected, as the target variable was not scaled. These results indicate that the model is able to capture general trends in daily bike rental demand, while leaving a substantial portion of the variance unexplained.

After applying non-linear feature engineering, the training MSE decreases to 455,171.5057, indicating that the augmented feature set improves the model’s ability to fit the training data. In addition to the required analysis, we also evaluate performance on the test set. The test MSE before feature engineering is 658,413.2054, while after feature engineering it decreases to 583,799.8210, showing a reduction in prediction error on unseen data for this feature configuration.

Although the test performance improves after feature engineering, the magnitude of improvement is noticeably smaller than that observed on the training set. This suggests that the additional non-linear features primarily enhance the model’s ability to fit the training data, while only part of this increased capacity translates into better generalization. Some of the patterns captured by the engineered features may be specific to the training data and do not consistently generalize to unseen samples. In the absence of regularization, increasing model complexity does not guarantee proportional improvements in test performance.

Overall, the comparison between the baseline and feature-engineered models shows that simple non-linear transformations can significantly affect training error, while their impact on generalization remains more limited. These results highlight the trade-off between model capacity and generalization in linear regression.

6 Discussion and Conclusion

In this project, we applied linear regression to a real-world dataset following a complete preprocessing, modeling, and evaluation pipeline. Categorical features were encoded into numerical representations and continuous features were standardized prior to model training. These steps are required for applying the analytical closed-form least-squares solution and enable stable computation of model parameters. The resulting models train successfully and produce finite predictions and mean squared error values on both the training and test sets.

The experimental results show that the baseline linear regression model is able to capture general trends in daily bike rental demand, but its performance remains limited in explaining all observed variability. After introducing non-linear feature engineering, training error decreases, indicating that the expanded feature set increases the model’s capacity to fit the training data. The test error also decreases, showing that the model captures more relevant patterns; however, the improvement on the test set is smaller compared to the training set.

These observations highlight the trade-off between model capacity and generalization. While increasing complexity helps reduce bias, it also introduces a higher risk of overfitting, where the model begins to memorize noise in the training data. Future work could investigate regularization techniques [4] to control model complexity or explore more expressive models to better capture such relationships without sacrificing generalization. Additionally, more systematic feature engineering and validation strategies may further improve model performance.

7 Statement of Contributions

Yuhui Wu was responsible for Task 1, including data loading, preprocessing, and exploratory analysis. Zhibo Wang was responsible for Task 2, which involved implementing the linear regression model and evaluation pipeline. Yuran Wang was responsible for Task 3, focusing on non-linear feature engineering and related experiments. The report was jointly written by all three members based on the parts they completed.

8 Statement on the use of LLMs and other resources

Some portions of the written explanation in this project were assisted by AI for grammar checking and academic language polishing. In addition, some factual academic perspectives were inspired by AI, while all modeling, implementation, and experimental results were independently conducted by the authors.

Some discussion of test results was conducted with other groups. Although different random seeds were used, we expected the results to be within the same order of magnitude in order to avoid major implementation errors.

References

- [1] Dua, D. and Graff, C., “Bike sharing dataset.” <https://archive.ics.uci.edu/ml/datasets/Bike+Sharing+Dataset>, 2019.
- [2] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O’Reilly, 2019.
- [3] A. Apicella, F. Isgrò, and R. Prevete, “Don’t push the button! exploring data leakage risks in machine learning and transfer learning,” *Artificial Intelligence Review*, vol. 58, p. 339, 2025.

- [4] S. M. Kakade, S. Shalev-Shwartz, and A. Tewari, “Regularization techniques for learning with matrices,” *Journal of Machine Learning Research*, vol. 13, pp. 1865–1890, June 2012.